

Experiment 1: JSX and Virtual DOM

App.js

```
import React, { useState } from "react";

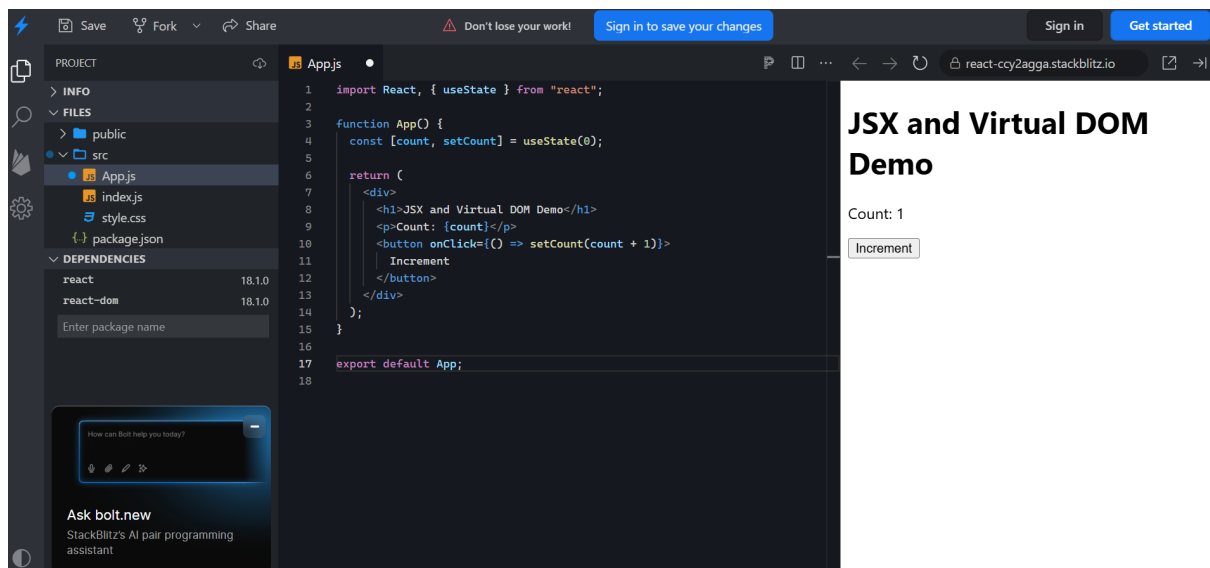
function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>JSX and Virtual DOM Demo</h1>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}

export default App;
```

Output

- Displays count value.
- Count updates instantly when button is clicked.



Experiment 1: React Application to Demonstrate JSX and Virtual DOM

Aim

1. To understand the use of JSX in React.
2. To demonstrate how the Virtual DOM updates UI efficiently.

Procedure

1. Create a React application using create-react-app.
2. Create components using JSX syntax.
3. Display dynamic data using state and props.
4. Observe UI updates when state changes.

Input

1. User clicks a button to change state.
2. Text or data entered by the user.

Output

1. JSX-rendered content displayed in the browser.
2. Updated UI without full page reload using Virtual DOM.

Experiment 1: Demonstrate JSX and Virtual DOM

Aim

To build a React application that demonstrates JSX and Virtual DOM concepts.

Software Requirements

- Node.js
- React.js
- Visual Studio Code
- Web Browser

Theory

JSX (JavaScript XML) allows writing HTML-like syntax within JavaScript. React uses a Virtual DOM to efficiently update only the changed parts of the user interface, improving performance.

Algorithm

1. Create a React application.
2. Create a state variable using useState.
3. Display the state value using JSX.
4. Add a button to update the state.
5. Observe the UI update through Virtual DOM.

Program

(Experiment 1 code here.)

Sample Input

Click the **Increment** button.

Sample Output

JSX and Virtual DOM Demo

Count: 0

[Increment]

After Click:

Count: 1

Result

Thus, a React application demonstrating JSX and Virtual DOM was developed and executed successfully.
